AI Engineering and Claude Architect · Part IV, Java

AI in Java, Lesson 3

A Production Agent Loop and Tools in Java

The agent worked until the day a retry arrived after a timeout. The first refund had gone through, the response had been lost, and the loop ran the tool again. The customer got paid twice, and every log looked normal.

Module	AI08 of 13
Lesson	3 of 8
Prerequisites	AI06 L2, L3, L5, L8, AI08 L1
Exam relevance	Java is out of scope. Implements TS 1.1, 1.4, 1.5, 2.2
Reading time	~40 minutes

Contents

1. A Production Agent Loop

Lesson 3 of 8 · AI in Java

A Production Agent Loop

Key idea

AI06 L2's loop is about ten lines. What makes it production code is everything around them: limits, a policy check before each tool, an idempotency ledger, safe error messages, and replaying each model reply unchanged. None of these asks the model to behave. They hold on a good day and a bad one, and each has a test that passes with a scripted model.

> *Before: the loop, tool design, tool errors and hooks in AI06. Teaches: those ideas as working Java with tests. Used by: AI08 L8.*

The hook: guarantees you can run

All real output from `AgentServiceTest`, using real services and a real H2 ledger. **The "model" is a fixed script**, so this tests our guardrails, and doesn't predict how Claude would behave.

```
agent sees: {"errorCategory":"business","isRetryable":false,
             "message":"Identity isn't verified. Call get_customer first."} isError=true
agent sees: {"verified":true} isError=false
agent sees: {"refunded":true} isError=false
agent sees: {"errorCategory":"business","isRetryable":false,
             "message":"Refunds over 500.00 need a human. Use escalate_to_human."} isError=true
refunds issued: [7781:40.00]
ledger denied: 2 ok: 1

same tool_use_id twice, refunds issued: 1, second result: {"refunded":true}
crash left a call unfinished: refunds issued=0, model sees: Already in progress.

tool threw 'orders-db password rejected at 10.2.3.4'
model sees: {"errorCategory":"transient","isRetryable":true,"message":"The tool failed."}

stuck model: STEP_LIMIT after 25
huge context: TOKEN_LIMIT after 2 steps, tokens=300200
truncated: TRUNCATED, refunds=0
```

Line breaks are added. **Compare the third group with AI07 L4**, where both frameworks passed that same exception text to the model by default.

Intuition: a pharmacy

A doctor (the model) writes prescriptions. The pharmacist (your code) has rules that no doctor can override. Check the patient's identity. Refuse a dose above the limit. Record each dispensing in a ledger, so a duplicate prescription isn't filled twice. And if the robot breaks, don't read the service engineer's notes to the patient.

The doctor is very good, and the rules exist anyway.

Mechanism: the shape of the service

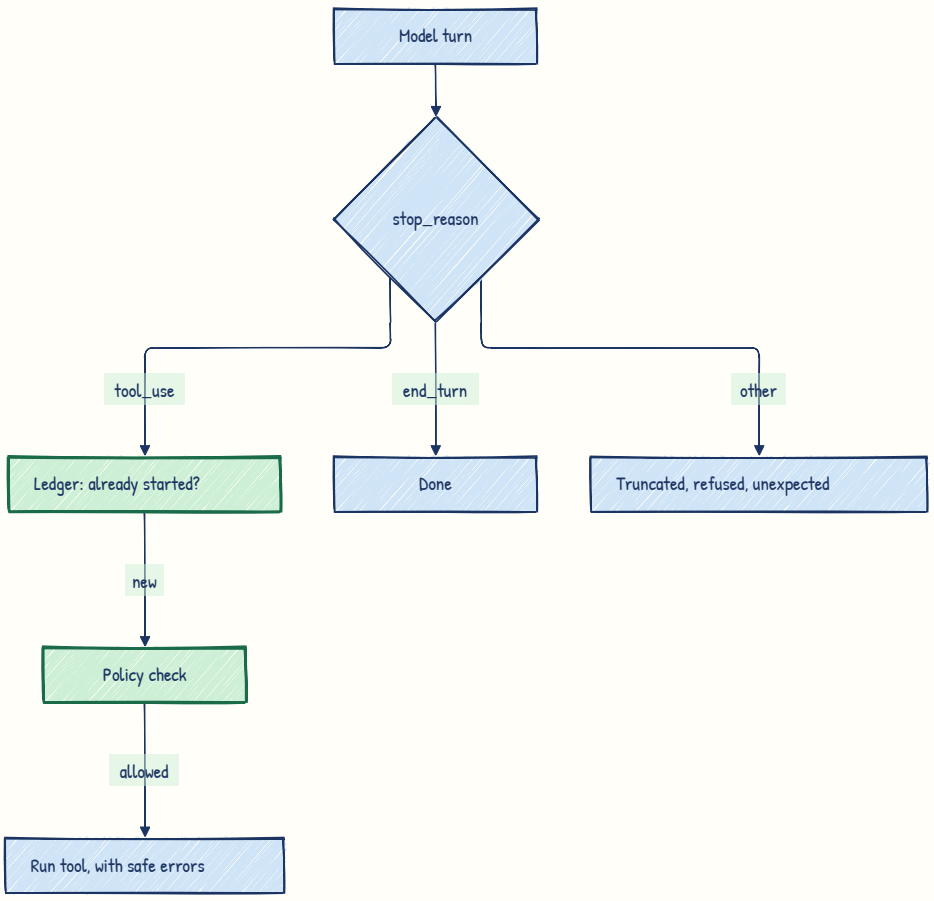


Figure 1. One iteration, with the parts that make it production code. The policy and the ledger run on every call, regardless of what the model asks.

The loop

From `AgentService.java`:

```
switch (turn.stopReason()) {
    case "end_turn" -> {
        return new Result(Status.DONE, turn.text(), step, tokens);
    }
    case "tool_use" -> {
        history.add(new ModelPort.Assistant(turn));
        history.add(new ModelPort.ToolResults(runAll(turn.calls(), context)));
    }
    case "pause_turn" -> history.add(new ModelPort.Assistant(turn));
```

Every exit names what happened: `DONE`, `TRUNCATED`, `REFUSED`, `STEP_LIMIT`, `TOKEN_LIMIT`, `UNEXPECTED`. So a caller can't mistake a limit for an answer, unlike the Spring AI behavior seen in AI07 L4.

Replay the reply unchanged

On Opus 5.5, thinking is always on, and thinking blocks must come back unchanged. So the port carries the SDK's message as an opaque value. Real output from the second request, against the fake API:

```
second request messages: [
  {"content":"Where is 7781?","role":"user"},
  {"content":[{"signature":"sig-abc","thinking":"","type":"thinking"},
              {"id":"toolu_1","input":{"order_id":"7781"},"name":"lookup_order","type":"tool_use"}],
   "role":"assistant"},
  {"content":[{"tool_use_id":"toolu_1","type":"tool_result","content":"delivered",...}],"role":"user"}]
```

```
case Assistant assistant
        -> builder.addMessage((Message) assistant.turn().raw());
```

A clean-looking port that rebuilt messages from text and tool calls would drop thinking blocks, which is one way that history-editing errors happen on current models. So history stays append-only.

The ledger: idempotency

```
if (!ledger.begin(call.id(), context.caseId(), call.name(), input)) {
    String earlier =
            ledger.previousResult(call.id()).orElse("Already in progress.");
    return new ToolResult(call.id(), earlier, false);
}
```

Situation	Result
New ID	Runs
Same ID, finished	Returns the stored result, with no second refund
Same ID, unfinished	**Doesn't run.** Reports "in progress"

That last row is the hard one. A crash between `begin` and `finish` leaves you not knowing whether the refund happened. Not re-running is the safe default for money, but someone or something must resolve it: query billing with the same key, and reconcile. Writing a first test of this found a real bug: my first version threw a `NullPointerException` on the unfinished row.

Pass the same key to the backend. The ledger protects this service, while the billing service's own idempotency protects everything.

Policy

A correction from later in the course. AI09 L3 attacks this policy with a scripted compromised model. It checks identity and a limit, and it does not check whose order this is, whether the amount is positive, or the running total. AI09 L3 shows the defects and a hardened version, so read this as the structure, and not as a complete rule set.

```
if (!verified.contains(context.caseId())) {
    return new Deny("Identity isn't verified. Call get_customer first.");
}
BigDecimal amount = new BigDecimal(String.valueOf(call.input().get("amount")));
if (amount.compareTo(limit) > 0) {
```

The denial goes back as a tool error, and the message tells the model what to do next. `verified` is set in code, from the tool's actual result:

```
if (call.name().equals("get_customer") && result.contains("\"verified\":true")) {
```

Two limits of this sketch. It holds `verified` in memory, so a restart loses it, and it should be in the case record. And `contains` on JSON is crude, so parse a typed result in real code.

Safe errors

Tools throw `ToolFailure` with a message that's safe to share. Anything else is treated as a crash, and the detail stays in our logs:

```
} catch (RuntimeException e) {
    // The detail stays in our logs. It could hold hosts, SQL or personal data.
    LOG.error("Tool {} threw", call.name(), e);
    ledger.finish(call.id(), "CRASHED", "");
    return failure(call, AgentTool.Category.TRANSIENT, true, "The tool failed.");
}
```

Parallel calls, in order

Real output, three calls in one turn, returned together in the original order:

```
parallel: t-a false {"found":true,"status":"DELIVERED","total":40}
parallel: t-b false {"found":false}
parallel: t-c true {"errorCategory":"validation","isRetryable":false,
                    "message":"order_id must be digits only, such as 7781"}
```

Notice `t-b`: no match isn't an error (AI06 L5).

Parallel calls and order. If the model asks for `get_customer` and `process_refund` together, they run at once, so the gate may or may not see verification in time. It stays safe either way, because the refund is refused unless verification has finished. If order matters for correctness, `disable_parallel_tool_use` is available.

Identity isn't a parameter

`AgentTool.Context` holds case and customer IDs set by our code, and isn't part of any tool schema. So the model can't choose whose data to read (AI08 L8).

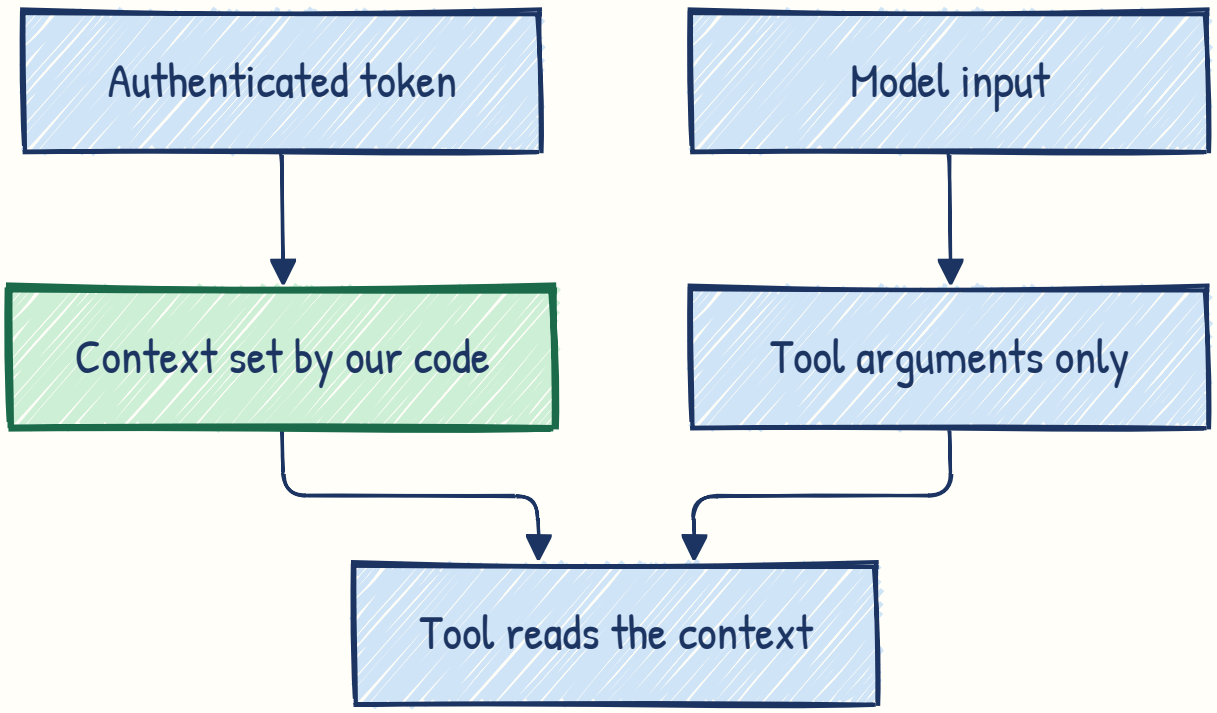


Figure 2. Where identity comes from. The token sets the context in code, and the tool reads it there, so a model-supplied value has no way in.

Why not use the Tool Runner or Spring AI?

Option	Fit
Tool Runner (beta)	Good for most agents. Gate inside the tool's run function
Spring AI	Good in Spring, with `ToolContext`. Set error handling, and check limit behavior
Own loop (here)	Used because this course shows every part. You own the edge cases

I'd probably start with one of the first two in a real service, and move only when something gets in the way. That's a judgment, not a measurement.

Why the obvious fixes fall short

Tempting move	Why it fails	Better
Rely on the API not to repeat	Your retries and crashes repeat calls	A ledger keyed by `tool_use` ID
Catch all, and send `e.getMessage()`	Leaks internals	Typed failures, and generic text
Limit steps only	One huge context can cost more than 25 small steps	A token budget too
Rebuild history from text	Loses thinking and tool blocks	Replay the whole message
Rules in prompts	Probabilistic	Policy in code
Assume verification from the model	It could be wrong, or injected	Set it from the tool's result

How Anthropic approaches it

Point	Source
Run all tools, and return all results in a single user message	Claude API reference
Validate inputs, and gate destructive operations	Same
Return `is_error` with an informative message	Same
Append the full `response.content`, or lose blocks	Same
Dedicated tools let the harness gate, audit and parallelize	Agent design guide

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: the loop in a controller.

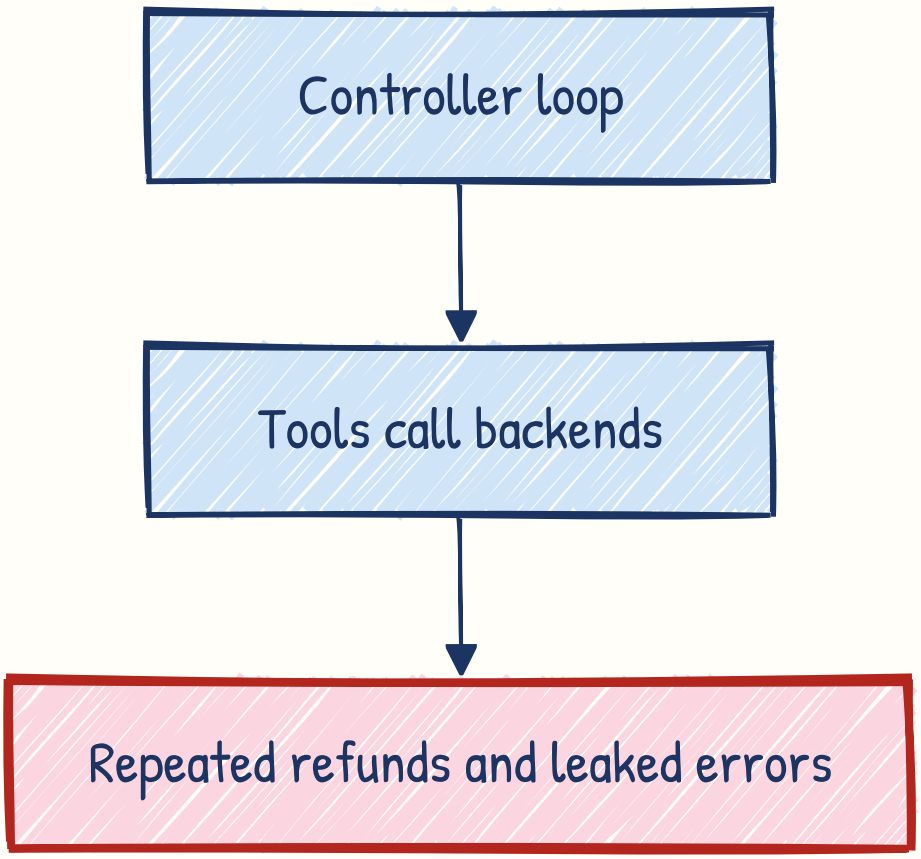


Figure 3. Baseline. A loop with no limits or ledger, and tools that talk straight to backends.

Improved: this module's service. Rules hold, and repeats are stopped. New problem: **state lives in one process, so a restart loses the run, and long runs tie up a request.**

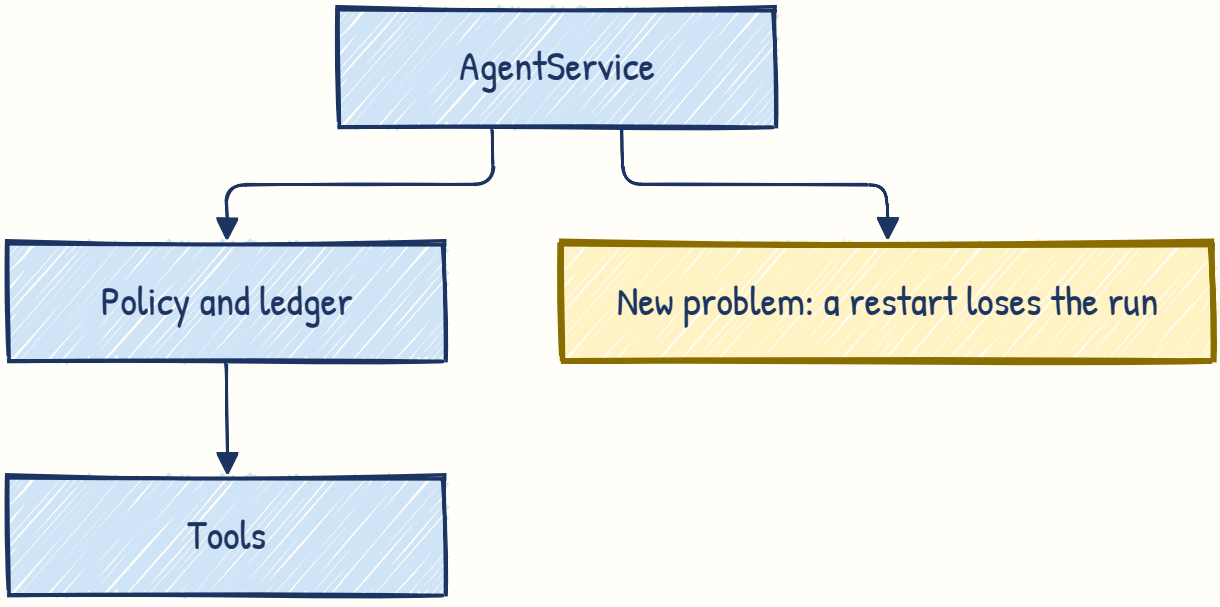


Figure 4. Improved. Policy and ledger protect each call, but the run itself lives in one process.

Production: durable runs.

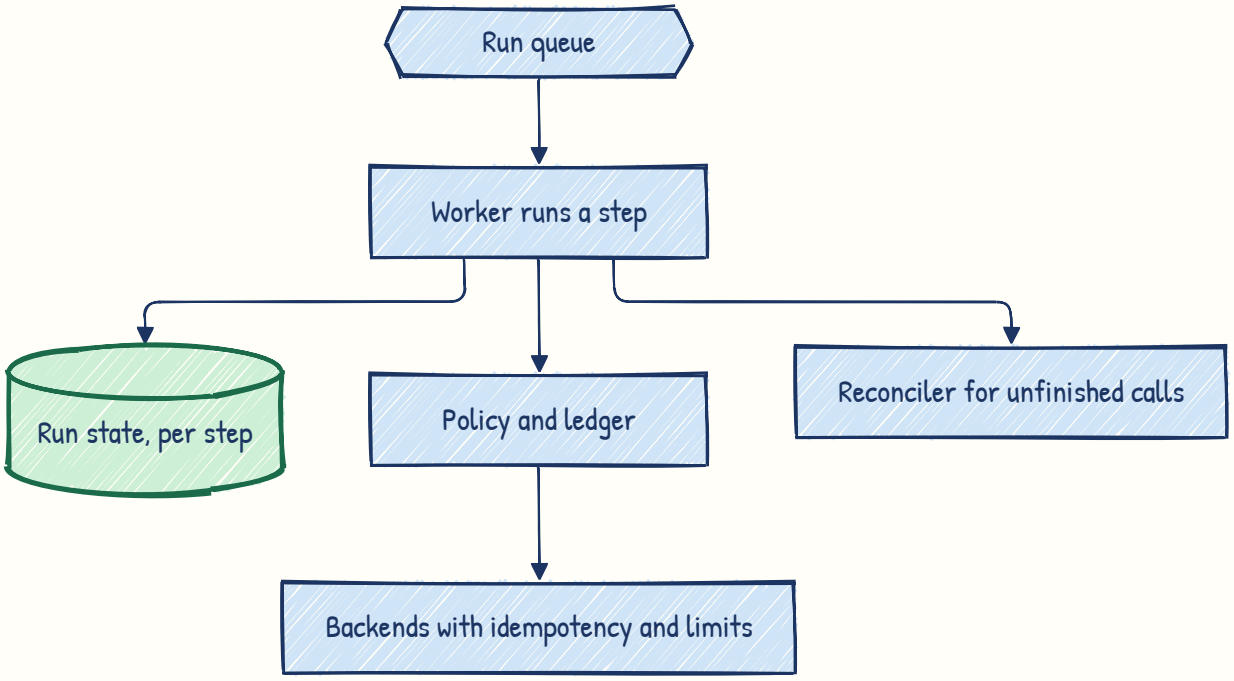


Figure 5. Production. Each step is saved, so another worker can resume a run. Backends enforce limits themselves, using the same idempotency key.

Component	Why it exists	How it communicates
Run state	Resume after a crash	Database, append-only history
Ledger	Idempotency and audit	SQL, unique key
Reconciler	Resolves unfinished calls	Scheduled, queries backends
Backends	The real boundary	Own limits, same key

Note that `raw` needs persisting. Saving run state means serializing SDK messages, which this module doesn't do yet, so test that a saved message replays byte for byte.

Failures

Failure	Containment
Crash mid-tool	"In progress", plus a reconciler
Stuck model	Step limit, named as such
Huge context	Token budget, named as such
Truncated turn	Never run its tools
Tool throws	Generic message, with detail logged
Policy service down	Fail closed

Trade-offs

Decision	Gain	Cost
Own loop	See and test everything	You own every stop reason
Virtual thread per tool call	Parallel, simple code	Limits needed for backends
Ledger in the same database	Simple, transactional	Coupled to that database
Token budget	Bounds cost	May stop a valid long run

When simpler is fine: read-only tools with no side effects don't need a ledger.

Common mistakes

1. **No idempotency.** Retries repeat side effects.
2. **Re-running an unfinished call.** It may have succeeded.
3. **Only a step limit.** Also limit tokens.
4. **Sending exception text to the model.** Log it instead.
5. **Rebuilding assistant messages.** Replay them whole.
6. **Holding verification in memory.** It's lost on restart.
7. **Parsing JSON with `contains`.** It's a sketch here, so parse properly.
8. **Treating a limit as an answer.** Name it, and handle it.

What to say in an interview

"The loop is short, and the production work is around it. A ledger keyed by the tool_use ID stops repeats, and an unfinished call isn't rerun, but goes to a reconciler. Policy checks run in code on every call, and identity comes from the token, never from tool input. Tool crashes return generic text. Every exit is a named status, so a limit can't pass for an answer. I'd also replay model messages unchanged, since thinking blocks must survive."

Certification lens

How this lesson maps to CCAR-F

Java isn't tested, but this implements **TS 1.1** (loop), **TS 1.4** and **TS 1.5** (gates and interception), and **TS 2.2** (structured errors).

These are original practice questions for understanding, not real or leaked exam questions.

Q Q1 · Code analysis · Hard. This runs a tool: `var r = tool.run(input); ledger.save(id, r);`. Name two failure cases, and redesign it.

A A crash after `run` and before `save` means a retry repeats the effect. Two concurrent attempts can both run. Redesign: claim the ID first with a unique insert, run only if the claim succeeded, then save the result. Treat an unfinished claim as unknown, reconcile it, and pass the ID to the backend as its idempotency key.

Q Q2 · Debugging · Medium. After a refactor, requests fail with a 400 on the second turn only: a thinking block's signature is invalid. What's the likely cause?

A The history is being edited or rebuilt, so the thinking block no longer matches, or it's been dropped from the assistant message. Keep history append-only, and replay the whole original message. Check for changes to earlier messages, the system prompt or the tool list too, since those can trigger it.

Q Q3 · Best decision · Medium. A tool needs the customer's ID. Should it be a parameter in the schema? Why or why not?

A No. The model would fill it in, and could be misled by injected text into reading someone else's data. Set it in code from the authenticated session, and pass it as context, outside the schema. Then a model-supplied value can't have any effect.

Revision - Lesson 3

→ The loop is short. Production work is limits, policy, ledger, safe errors, and replay.

→ Every exit is a named status, so limits can't pass as answers.

→ Replay the whole model message, including thinking and tool_use blocks.

→ History stays append-only.

→ Claim the tool_use ID first. Same ID, finished: return the stored result.

→ Same ID, unfinished: don't run. Reconcile.

→ Pass the same key to the backend.

→ Policy runs in code before every tool.

→ Set verified state from the tool's result, not from the model.

→ Tool crashes return generic text, with detail in logs.

→ Identity comes from the token and context, never from tool input.

→ Limit both steps and tokens.

Your notes